

TECNOLÓGICO SUPERIOR DE JALISCO

CAMPUS LA HUERTA

TRABAJO:

ACTIVIDAD 2_INVESTIGACION

ASIGNATURA:

LENGUAJE DE INTERFAZ

CARRERA:

INGENIERÍA EN SISTEMAS
COMPUTACIONALES, ISIC
2010-224

ALUMNO:

JAIME RAMIREZ

NUMERO DE CONTROL:

230111657

PROFESOR:

AGUSTIN MEDINA BAUTISTA

LA HUERTA, JALISCO. 11/02/2026

Indice

INFORME DE INVESTIGACION	3
1. Introducción	3
2. Desarrollo	11
2.1. <i>Llamadas a Servicios del Sistema</i>	11
2.2. <i>Modos de Direccionamiento</i>	12
3. Conclusión	13
4. Bibliografía	14

INFORME DE INVESTIGACION

1. Introducción

El presente documento describe la estructura, funcionamiento e implementación del sistema desarrollado en lenguaje ensamblador para entorno DOS. El programa permite la interacción con el usuario mediante consola, realizando distintas operaciones numéricas y mostrando resultados de manera dinámica.

El sistema incluye:

- Captura de nombre del usuario
- Menú interactivo
- Conversión de números
- Clasificación de números
- Cálculo de raíz cuadrada mediante el método babilónico

2. Estructura General del Programa

El programa está organizado en tres segmentos principales:

1.1. Segmento de Pila

pila segment para stack 'stack'

db 1024 dup(?)

pila ends

Se encarga de almacenar direcciones de retorno, variables temporales y apoyo en procesos como impresión de números.

1.2. Segmento de Datos

Contiene:

- Cadenas de texto para interacción con el usuario
- Variables de almacenamiento (num, aprox)
- Buffer para el nombre

Ejemplo:

nombre db 30 dup('\$')

num dw ?

aprox dw ?

2.3 Segmento de Código

Contiene la lógica del programa, organizada en:

Procedimiento principal (compa)

Subrutinas auxiliares:

- Lectura de número
- Impresión de número
- Conversión
- Clasificación
- Raíz cuadrada

3. Flujo del Programa

3.1 Inicio

Inicialización de segmentos (DS, ES)

Limpieza de pantalla

Captura del nombre del usuario

3.2 Interfaz principal

Se muestra un mensaje personalizado:

Bienvenido - [Nombre] - seleccione la operación a realizar

3.3 Menú

El sistema presenta opciones mediante un ciclo:

- Conversión de números
- Clasificación de números
- Raíz cuadrada
- Salida

El programa permanece en ejecución hasta que el usuario decide salir.

4. Entrada de Datos

La captura de números se realiza mediante la rutina:

leer_numero proc

Funcionamiento:

- Lee carácter por carácter
- Convierte de ASCII a valor numérico
- Aplica la fórmula:

$\text{numero} = \text{numero} * 10 + \text{digito}$

Esto permite construir números de hasta 5 dígitos (0–65535).

5. Conversión de Números

El sistema permite la conversión de números a diferentes sistemas numéricos.

Generalidades:

- Se utiliza división sucesiva
- Se almacenan residuos en la pila
- Se imprimen en orden inverso

Sistemas contemplados:

- Binario
- Octal

- Hexadecimal

(La implementación sigue la misma lógica con diferentes bases numéricas.)

6. Clasificación de Números

El programa determina si un número es:

- Perfecto
- Abundante
- Deficiente

Enfoque general:

- Se calculan divisores propios del número
- Se suman
- Se comparan con el valor original

No se profundiza en la implementación, pero se basa en operaciones iterativas y divisiones.

7. Raíz Cuadrada (Método Babilónico)

El módulo de raíz cuadrada del sistema implementa el método babilónico, también conocido como el método de aproximaciones sucesivas. Este algoritmo es una técnica numérica que permite obtener una aproximación de la raíz cuadrada de un número entero positivo sin necesidad de funciones matemáticas avanzadas.

7.1 Principio del algoritmo

El método se basa en la siguiente fórmula iterativa:

$$\text{aprox} = (\text{aprox} + (N / \text{aprox})) / 2$$

Donde:

- N: número ingresado por el usuario

- **aprox:** valor estimado de la raíz
- **N / aprox:** mejora de la estimación

El proceso se repite varias veces hasta que la aproximación converge a un valor cercano a la raíz real.

7.2 Implementación en el programa

En el sistema, el valor inicial de la aproximación se obtiene mediante una división simple:

```

mov             ax,             num
mov             cx,             2
div             cx
mov aprox, ax

```

Esto establece:

$\text{aprox inicial} = N / 2$

7.3 Proceso iterativo

El refinamiento de la raíz se realiza mediante un ciclo controlado por el registro CX, el cual define el número de iteraciones:

```

mov             cx,             5
iteracion:
    mov         ax,             num
    mov         bx,             aprox
    div         bx
    add         ax,             aprox
    shr         ax,             1
    mov         aprox,         ax
loop iteracion

```

7.4 Descripción del proceso paso a paso

En cada iteración ocurre lo siguiente:

1. Se toma el valor original N
2. Se divide entre la aproximación actual (aprox)
3. Se suma el resultado con la misma aproximación
4. Se divide entre 2 utilizando un desplazamiento lógico (SHR)
5. Se actualiza la variable aprox con el nuevo valor

7.5 Uso del desplazamiento (SHR)

En lugar de realizar una división convencional entre 2, el programa utiliza:

```
shr ax, 1
```

Este operador realiza un **desplazamiento lógico a la derecha**, equivalente a dividir entre 2.

Ventajas del uso de SHR:

- Mayor eficiencia que DIV
- Menor consumo de ciclos de CPU
- Simplificación del código
- Adecuado para operaciones de enteros positivos

7.6 Convergencia del resultado

El método babilónico converge rápidamente, por lo que con pocas iteraciones (en este caso 5) se obtiene una aproximación suficientemente precisa para números enteros dentro del rango permitido (0 a 65535).

7.7 Ejemplo conceptual

Para un número $N = 400$:

- Iteración 1: $\text{aprox} = 200$
- Iteración 2: $\text{aprox} \approx 100$
- Iteración 3: $\text{aprox} \approx 50$
- Iteración 4: $\text{aprox} \approx 25$
- Iteración 5: $\text{aprox} \approx 20$

Resultado final: aproximación estable de la raíz.

8. Salida de Datos

Los números se imprimen mediante una rutina que:

- Divide entre 10 repetidamente
- Guarda residuos en la pila
- Convierte a ASCII
- Imprime en orden correcto

Esto permite mostrar números completos sin importar su tamaño dentro del rango permitido.

9. Control de Flujo

El programa utiliza:

- CMP para comparaciones
- JE, JNE, JA, JB para decisiones
- LOOP para ciclos
- Esto permite una ejecución estructurada y controlada.

10. Consideraciones Técnicas

- Rango de números: 0 a 65535
- Uso de registros de 16 bits (AX, BX, CX, DX)
- Dependencia de interrupciones:
- INT 21H → Entrada/Salida
- INT 10H → Control de pantalla

11. Conclusión

El sistema implementa correctamente múltiples funcionalidades utilizando lenguaje ensamblador, demostrando:

Manejo de memoria y registros

Uso de pila

Interacción con el usuario

Aplicación de algoritmos matemáticos

Se trata de un programa estructurado, funcional y extensible, adecuado para demostrar el uso práctico del ensamblador en operaciones numéricas.

Programar en lenguaje ensamblador implica interactuar directamente con el hardware, sin las capas de abstracción que ofrecen lenguajes de alto nivel como Python o Java. Según ITTLenguajesdeInterfaz (s.f.), este tipo de programación permite un control detallado sobre cada operación del procesador, lo que lo convierte en una herramienta poderosa para comprender cómo funcionan internamente las computadoras y cómo el software se comunica con el hardware.

Para que un programa en ensamblador funcione de manera práctica, necesita dos capacidades principales: la primera es comunicarse con el sistema operativo y los periféricos, ya que la

CPU no puede realizar por sí solas tareas complejas como leer del teclado o escribir en la pantalla. La segunda es conocer con precisión dónde se encuentran los datos en la memoria y cómo manipularlos, utilizando los distintos modos de direccionamiento disponibles en la arquitectura del procesador (Tanenbaum y Bos, 2015).

El conocimiento de estos temas no solo permite escribir programas funcionales y eficientes, sino que también proporciona una comprensión más profunda de la interacción entre software y hardware, facilitando la optimización de recursos y la seguridad en la ejecución de instrucciones.

3. Desarrollo

3.1. Llamadas a Servicios del Sistema

En ensamblador, el procesador por sí solo no puede acceder directamente a la mayoría de los recursos del sistema. Para interactuar con el sistema operativo se utilizan las llamadas al sistema, que son solicitudes formales que permiten ejecutar tareas que requieren privilegios especiales (Irvine, 2014).

El proceso típico de una llamada al sistema funciona de la siguiente manera: primero, se cargan valores específicos en registros de la CPU, indicando la operación a realizar. Por ejemplo, en arquitecturas Intel, el registro EAX puede contener el código de función, mientras que otros registros, como EBX o ECX, pueden contener parámetros adicionales. Luego se ejecuta una instrucción de interrupción por software, como INT 21h en DOS o INT 80h en Linux, que transfiere el control al núcleo del sistema operativo (ITT Lenguajes de Interfaz, s.f.). La CPU detiene temporalmente la ejecución del programa, el sistema operativo realiza la tarea y finalmente devuelve el control al programa.

Algunos ejemplos de llamadas al sistema incluyen:

- Manejo de archivos: open, read, write, close. Permiten abrir, leer, escribir y cerrar archivos de manera segura.
- Gestión de procesos: fork, exec, exit, wait. Se usan para crear, ejecutar o finalizar procesos.

- Interacción con periféricos: permite la lectura de teclado o escritura en pantalla, garantizando que el programa pueda comunicarse con el usuario (Tanenbaum y Bos, 2015).

Estas llamadas aseguran que los programas no accedan directamente al hardware, protegiendo la estabilidad del sistema y evitando errores graves. Además, proporcionan un marco estandarizado para que el software funcione correctamente en distintos entornos.

3.2. Modos de Direccionamiento

El modo de direccionamiento define cómo el procesador localiza los datos necesarios para ejecutar una instrucción. Según Tanenbaum y Bos (2015), la elección del modo de direccionamiento afecta directamente la velocidad y eficiencia del programa.

Algunos de los modos de direccionamiento más relevantes son:

- Inmediato: el dato está contenido directamente en la instrucción. Por ejemplo, `MOV AX, 5` asigna el valor 5 al registro AX sin acceder a memoria adicional. Es el más rápido.
- Registro: el operando se encuentra en un registro de la CPU, como en `MOV AX, BX`. Los registros son memorias internas de alta velocidad.
- Directo (a memoria): la instrucción indica la dirección exacta de la memoria, como `MOV AX, [1000]`. Esto es útil para variables globales o celdas específicas.
- Indirecto e indexado: se utiliza un registro como puntero a la ubicación de memoria y se le puede sumar un desplazamiento, por ejemplo, `MOV AX, [BX+4]`. Es ideal para recorrer arreglos o estructuras de datos.
- Relativo al contador de programa (PC): se usa en instrucciones de salto, calculando la dirección destino en función de la posición actual.
- Apilado (stack): el operando se encuentra en la cima de la pila, gestionada con `PUSH` y `POP`. Esto permite un manejo ordenado de subrutinas y parámetros (Irvine, 2014).

El correcto uso de estos modos permite que el programa sea más eficiente y seguro, ya que un modo incorrecto puede provocar accesos indebidos a memoria o ralentizar la ejecución de instrucciones.

4. Conclusión

En esta actividad comprendí que las llamadas al sistema y los modos de direccionamiento son esenciales para programar en ensamblador de forma eficiente y segura. Las llamadas al sistema permiten interactuar con recursos del sistema operativo, mientras que los modos de direccionamiento determinan cómo se acceden y manipulan los datos en memoria.

Estos conceptos me ayudaron a entender que la programación en ensamblador no consiste solo en escribir instrucciones, sino en planificar cuidadosamente cómo se almacenan y procesan los datos. Esto refuerza que la verdadera fortaleza del ensamblador no está en su simplicidad, sino en el control preciso que ofrece sobre la ejecución de instrucciones y el uso de recursos del hardware.

Dominar estos temas permitirá desarrollar programas más confiables, eficientes y compatibles con distintos sistemas operativos, además de brindar una comprensión más profunda de la relación entre software y hardware.

5. Bibliografía

- ITTLenguajesdelInterfaz. (s.f.). *Importancia de la programación en lenguaje ensamblador*. Recuperado de <https://ittlenguajesdeinterfaz.wordpress.com/1-1-importancia-de-la-programacion-en-lenguaje-ensamblador/>
- Irvine, K. R. (2014). *Assembly language for x86 processors* (7th ed.). Recuperado de <https://openaccess.uoc.edu/server/api/core/bitstreams/7bb6c3ac-fd38-4b61-9b22-36a519e3d1bc/content>
- Tanenbaum, A. S., & Bos, H. (2015). *Sistemas operativos modernos* (4ª ed.). Recuperado de https://apps.utel.edu.mx/recursos/files/r161r/w24802w/Sistemas_Operativos_Modernos-ATanenbaum.pdf